Notes on Neurons and How They Train a Model

Luis Mendez

Question: How does a neuron work, and how do we use neurons to train a model?

Answer:

1. What a Neuron Computes

An artificial neuron is a loose model of a biological neuron: dendrites receive signals, the cell body combines them, and the axon fires an output if the combined signal is strong enough. The artificial version does the same thing with numbers.

Given an input vector $\mathbf{x} = (x_1, \ldots, x_m)$, a neuron holds one **weight** w_i per input and one **bias** b. It first forms a weighted sum,

$$z = \sum_{i=1}^{m} w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b,$$

then passes z through a nonlinear **activation function** ϕ to produce its output,

$$a = \phi(z).$$

- w_i: how strongly input x_i influences the neuron. Learned during training.
- b: shifts z up or down independently of the inputs (an always-on input, sometimes written as $w_0 x_0$ with $x_0 = 1$). Also learned.
- ϕ: fixed in advance (not learned); it decides *how* the weighted sum turns into an output.

2. Why the Activation Function Matters

If ϕ were the identity, stacking neurons into layers would still only compute a linear function of $\mathbf{x}$ — a composition of linear maps is itself linear, no matter how many layers are stacked. Nonlinear ϕ is what lets a network represent curved decision boundaries and complex relationships.

- **Sigmoid:** $\phi(z) = \dfrac{1}{1+e^{-z}} \in (0,1)$. Reads as a probability; common at the output layer for binary classification.
- **ReLU:** $\phi(z) = \max(0, z)$. Cheap, gradient is 0 or 1, the default for hidden layers.
- **tanh:** $\phi(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1,1)$. Like sigmoid but zero-centered.
- **Softmax:** generalizes sigmoid to multiple classes, producing a probability distribution over K outputs.

3. From One Neuron to a Network

A single neuron can only separate data with a straight line (or hyperplane). Layering neurons and connecting every neuron in one layer to every neuron in the next (a fully-connected, or *dense*, layer) lets the network build up complex features. For layer ℓ,

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{a}^{(\ell)} = \phi\Big(\mathbf{z}^{(\ell)}\Big),$$

with $\mathbf{a}^{(0)} = \mathbf{x}$. Running this input-to-output is **forward propagation**: it turns raw features into a prediction $\hat{y} = \mathbf{a}^{(L)}$. Each hidden layer re-represents the data; deeper layers combine simple patterns from earlier layers into more abstract ones.

4. Measuring How Wrong the Network Is

Training means choosing every $W^{(\ell)}, \mathbf{b}^{(\ell)}$ so that predictions $\hat{y}$ are close to true targets y. A **loss function** C quantifies the gap, e.g.

$$C = \frac{1}{2}(\hat{y}-y)^2 \quad \text{(regression)}, \qquad C = -\big(y \log p + (1-y)\log(1-p)\big) \quad \text{(bi}$$

where p is the sigmoid output. Training is the optimization problem of minimizing C, averaged over the training set, as a function of every weight and bias.

5. Gradient Descent: How Weights Actually Move

C is a nonconvex function of thousands (or millions) of parameters, so there is no closed-form minimizer. **Gradient descent** instead nudges every parameter opposite its gradient, the direction of steepest increase, so that C decreases:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \nabla_{\mathbf{w}} C,$$

where η is the **learning rate**. Too large an η overshoots and can diverge; too small makes training crawl and risks stalling in a shallow local minimum.

6. Backpropagation: Computing the Gradient Efficiently

Backpropagation computes $\nabla_{\mathbf{w}} C$ for *every* weight in one backward sweep, by reusing the chain rule layer by layer instead of recomputing each partial derivative from scratch. Define the error at layer ℓ as $\boldsymbol{\delta}^{(\ell)} = \partial C / \partial \mathbf{z}^{(\ell)}$. At the output layer,

$$\boldsymbol{\delta}^{(L)} = \frac{\partial C}{\partial \mathbf{a}^{(L)}} \odot \phi'\Big(\mathbf{z}^{(L)}\Big),$$

and each earlier layer's error is derived from the layer after it,

$$\boldsymbol{\delta}^{(\ell)} = \Big(\big(W^{(\ell+1)}\big)^\top \boldsymbol{\delta}^{(\ell+1)}\Big) \odot \phi'\Big(\mathbf{z}^{(\ell)}\Big).$$

Once every $\boldsymbol{\delta}^{(\ell)}$ is known, the weight and bias gradients fall out directly:

$$\frac{\partial C}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \left(\mathbf{a}^{(\ell-1)}\right)^{\top}, \qquad \frac{\partial C}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}.$$

Algorithm 1 Training one example

Forward pass: compute $\mathbf{z}^{(\ell)}, \mathbf{a}^{(\ell)}$ for $\ell = 1, \ldots, L$
Output error: $\boldsymbol{\delta}^{(L)} \leftarrow (\hat{y} - y) \odot \phi'(\mathbf{z}^{(L)})$
for $\ell = L-1, \ldots, 1$ **do**
    $\boldsymbol{\delta}^{(\ell)} \leftarrow \left((W^{(\ell+1)})^{\top} \boldsymbol{\delta}^{(\ell+1)}\right) \odot \phi'(\mathbf{z}^{(\ell)})$
end for
for $\ell = 1, \ldots, L$ **do**
    $W^{(\ell)} \leftarrow W^{(\ell)} - \eta\, \boldsymbol{\delta}^{(\ell)} (\mathbf{a}^{(\ell-1)})^{\top}$
    $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta\, \boldsymbol{\delta}^{(\ell)}$
end for

7. Batch vs. Stochastic vs. Mini-Batch

In practice, the gradient is not computed from a single example or the whole dataset at once:

- **Batch GD**: exact gradient over all n examples per update; accurate but slow, and deterministic enough to get stuck in a local minimum.
- **Stochastic GD (SGD)**: one example per update; noisy but cheap, and the noise helps escape local minima/saddle points.
- **Mini-batch GD**: a small batch (e.g. 32–256 examples) per update; the standard in practice, balancing gradient variance against compute cost and vectorizing well on a GPU.

8. Putting It Together

- A neuron = weighted sum + bias, squashed by an activation function.
- Stacking neurons into layers (forward propagation) turns raw inputs into a prediction.
- A loss function scores how wrong that prediction is.
- Backpropagation computes how each weight should change to reduce the loss.
- Gradient descent (batch, stochastic, or mini-batch) actually applies that change, repeated over many epochs, until the model fits the training data.

9. Typical Keras Example

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(m,)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=50, batch_size=
```

Each `Dense` layer is a bank of neurons; `fit` runs forward propagation, computes the loss, backpropagates the gradient, and applies mini-batch updates (via `adam`, an adaptive variant of gradient descent) once per batch, for the given number of epochs.